Project 1: Colorizing the Prokudin-Gorskii Photo Collection

Automatically aligning digitized glass-plate negatives into full-color images

OVERVIEW

The Problem

Back in 1907, color photography did not exist yet. However, the photographer Sergei Prokudin-Gorskii had a clever idea.

He knew that all the colors that humans could see were built from red, green, and blue. Instead of capturing color directly, he envisioned that if he could take 3 black and white photos of the same scene and apply a red filter to the first one, a green filter to the second one, and a blue filter to the third one, he could recreate the color image by stacking them on top of each other. Each black and white photo captures exactly how much of each color of that filter is present at each point in the image.

PART 1

Single-Scale Alignment

Stacking the photos on top of each other is not a trivial thing to do. Because the technology that could take 3 photos in quick succession didn't exist back then, there are slight alignment errors between each of the photos taken. For me to be able to restore the color image as accurately as possible, I must align the images the best I can.

The function that I can use to help me here is np.roll(). It slides all the pixels along the x and y axes by some specified amount, and wraps around any pixels that fall off.

Now that I am able to shift the images, the next step is to test how well the shifts (red

and green filters) align with the reference frame (the blue filter in this instance). There are two ways I can do this.

The first way is through the L2 norm (Euclidean distance). If I take the square root of the squared difference between pixel brightness values, I get a number that tells me roughly how similar the two images are, with a number closer to 0 being more similar.

The other method is through Normalized Cross Correlation (NCC). First I subtract the mean and normalize each of the image vectors, and then I take the dot product of both vectors. Because they have both been demeaned and normalized, if they are identical their dot product will be one. Therefore, the higher/closer the dot product is to 1, the more similar the two images are.

While both are good options, I chose to use NCC for my project because the L2 norm is very sensitive when there are brightness differences between the different RGB channels, whereas NCC is more robust for those instances.

When testing for which shift was best, I used a +-15 pixel window. This is because the number of shifts you have to check grows exponentially, so 15 is a happy medium where you are still checking a wide range of shifts, but it is still not that computationally expensive.



Cathedral
G: dx=[5], dy=[2] · R: dx=[12], dy=[3]



Monastery
G: dx=[-3], dy=[2] · R: dx=[3], dy=[2]



Tobolsk
G: dx=[3], dy=[3] · R: dx=[6], dy=[3]

PART 2

Multi-Scale Pyramid Alignment

While the single scale alignment method works for smaller .jpg files, I find that it becomes too computationally expensive for bigger .tif files. This is because the .tif files can have millions of pixels, so the sliding window of 15 pixels isn't enough. There are too many different shifts to check efficiently and even the ordinary pixel arithmetic adds up

once you're in the millions.

The fix to this is through image pyramids. By making the image smaller (e.g. removing every other row and column), you keep the image centered but reduce the number of pixels you have to deal with significantly. Image pyramids are a form of recursion where the base case is a certain pixel count that is manageable to perform single scale alignment on. To reach the base case, at each level the pixel count is reduced by a factor of two, and once the shift from the previous level is passed up, a smaller sliding window (4 pixels) is applied to better align the images. Once you reach the full resolution image again, the alignment has been refined at every level, and the small window at the end catches any last errors.



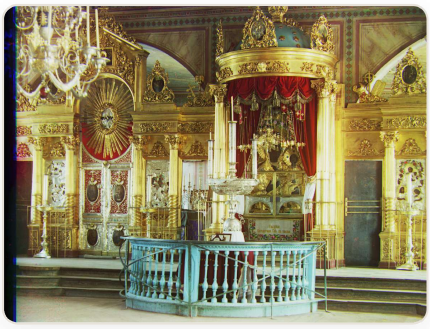
Church
G: dx=[25], dy=[4] · R: dx=[58], dy=[-4]



Emir
G: dx=[49], dy=[24] · R: dx=[141], dy=[-205]



Harvesters
G: dx=[60], dy=[17] · R: dx=[124], dy=[13]



Icon
G: dx=[41], dy=[17] · R: dx=[89], dy=[23]



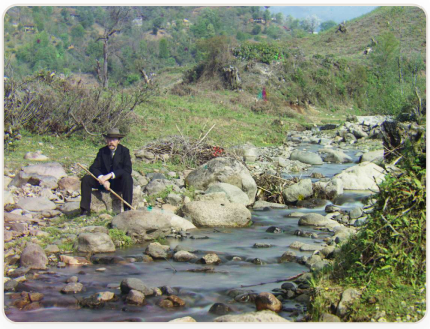
Ilemselga
G: dx=[40], dy=[7] · R: dx=[130], dy=[11]



Melons
G: dx=[82], dy=[11] · R: dx=[178], dy=[13]



Religous Painting
G: dx=[28], dy=[3] · R: dx=[68], dy=[7]



Self Portrait
G: dx=[79], dy=[29] · R: dx=[176], dy=[37]



Siren
G: dx=[49], dy=[-6] · R: dx=[96], dy=[-25]



Three Generations
G: dx=[53], dy=[14] · R: dx=[112], dy=[11]



Wharf
G: dx=[15], dy=[-7] · R: dx=[83], dy=[16]

PART 3

My Own Selections

These are three additional glass plates I downloaded from the **Library of Congress Prokudin-Gorskii collection** and ran through the same pipeline.



Camel
G: dx=[46], dy=[-3] · R: dx=[98], dy=[-20]



On the Island of Capri
G: dx=[45], dy=[-13] · R: dx=[103], dy=[-11]



In a Conservatory
G: dx=[60], dy=[28] · R: dx=[126], dy=[35]

BELLS & WHISTLES

Automatic Border Cropping

Rather than crop a fixed margin off every image, I detect the border here automatically per image. After aligning, I measure how much the three channels disagree at each row and column. Real photo content is well aligned and has low disagreement, while the scan border isn't real scene content in any channel, so it shows up as a spike. I can scan in from each edge and crop past the last row/column whose disagreement exceeds a

threshold relative to the image's interior, which correctly removes non-contiguous or uneven borders that a fixed-percentage crop would miss.



Before



After

Edge-Based Alignment

The issue that I have when trying to reconstruct the emir image is that when I run into parts of the image that have different pixel brightnesses for each of the filters (e.g. a part of his robe may be very dark in the blue filter but bright in the red filter), the L2/NCC alignment scores don't work well.

Sobel filters fix this issue. Instead of looking directly at pixel to pixel brightness comparisons, Sobel filters try to detect edges in the image. To detect an edge, they calculate how quickly brightness is changing at each pixel, and if in a row/column the brightness is changing rapidly, then an edge is detected.

To find how fast brightness is changing from pixel to pixel, Sobel filters use a kernel that encodes a pattern that reinforces edges and cancels out flat parts of the image. A convolution is then performed on the entire image to get a final edge magnitude image, which I can then run NCC on to find the shift that best aligns the edges of the images.



Raw pixel-value alignment

G: dx=[49], dy=[24] · R: dx=[141], dy=[-205]



Sobel edge-based alignment

G: dx=[49], dy=[24] · R: dx=[104], dy=[56]

← Back to portfolio

Alex Aney · CS180 · UC Berkeley